

Week 6: R functions

Candice Lei (PID: A18585167)

2026-04-16

Table of contents

Background	1
Write your first R function: add()	1
Write a generate_dna() function	3
Write a 'generate_protein()' function	5
Generate random protein sequences of length 6 to 13	6
BLASTp search against nr — are your peptides “unique in nature”?	7
Connecting your findings to immunology	8

Background

All functions in R have at least 3 things: - a *name* (we pick that and use it to call the function)
- input *argument* (one or more comma separated inputs that go inside the brackets when we call the function) - the *body* (the lines of R code that do the work of the function)

Write your first R function: add()

Here we will create a function to add some numbers. Let's call it 'add()'

Input arguments can be either **required** or **optional**. The later have fall back default values that will be used if the user does not specify them.

Q1a. Your first version of the function should add two input numbers together. For example, add(4,7) should return 11.

```
add <- function(x, y = 1000) {  
  x + y  
}
```

Can we use our new function:

```
add(10,1000)
```

```
[1] 1010
```

```
add(10)
```

```
[1] 1010
```

```
add(4,7)
```

```
[1] 11
```

Q1b. For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add( c(4,7,10) )`. [1 pt]

```
add <- function(x, y = 0) {  
  sum(x,y)  
}
```

```
add(4,7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the... (dots) argument or the base R function `sum()`. [2 pts]

```
add <- function(...) {  
  sum(...)  
}
```

```
add(1,2,3,-4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the ‘return()’ function call.

```
add <- function(...) {  
  return(sum(x,y,...))  
  cat("Is it break time yet?\n")  
}
```

Write a generate_dna() function

A useful function here is the “base R” ‘sample()’ function:

```
sample(1:5, size=3)
```

```
[1] 5 2 3
```

```
sample(1:5, size=60, replace = TRUE)
```

```
[1] 5 3 1 1 3 2 1 1 3 5 2 5 5 3 1 2 3 2 5 5 3 3 2 4 1 2 5 2 2 4 3 3 2 5 5 5 2 5  
[39] 4 3 5 2 5 5 1 5 1 1 2 2 2 2 5 2 5 5 1 4 2 5
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”...

```
sample(x=c("A","C","G","T"), size = 10, replace = TRUE)
```

```
[1] "A" "A" "T" "C" "G" "C" "G" "C" "C" "C"
```

Q2a. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
generate_dna <- function(len) {
  sample(x=c("A","C","G","T"), size = len, replace = TRUE)
}
```

```
generate_dna(6)
```

```
[1] "C" "T" "C" "T" "T" "T"
```

Q2b. Your second version should **optionally** be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

Functions that could be useful here are ‘paste()’, ‘if()’, ‘cat()’, and ‘return()’

```
generate_dna <- function(len=10) {
  sample(x=c("A","C","G","T"), size=len, replace = TRUE)
}
```

```
generate_dna <- function(len=10, single.element=TRUE) {
  ans <- sample(x=c("A","C","G","T"), size=len, replace = TRUE)
  # cat("Hello...")
  if(single.element) {
    # cat("is it me you are looking for...")
    ans <- paste (ans, collapse = "")
  }
  return(ans)
}
```

```
generate_dna(single.element = FALSE)
```

```
[1] "C" "C" "C" "G" "G" "T" "C" "T" "G" "T"
```

```
paste(c("A", "C", "G"), collapse = "---")
```

```
[1] "A---C---G"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example: >len9
CGAAGGCTG Be sure to include clear comments that explain each step of your function. [2 pts]

```
>len9
CGAAGGCTG
```

```
cat("hello \n there")
```

```
hello
  there
```

```
generate_dna <- function(len=10, single.element=TRUE) {
  ans <- sample(x=c("A","C","G","T"), size=len, replace = TRUE)

  if(single.element) {

    ans <- paste (ans, collapse = "")
  }
  ## Format as FASTA with an ID line
  cat( paste(">len", len, "\n", sep = ""))
  cat(ans)
  cat("\n")
  ##

  return(ans)
}
```

```
x <- generate_dna(20)
```

```
>len20
CTTTCAGCTACGGAAATCGT
```

Write a 'generate_protein()' function

Q3. Write a function 'generate_protein()' that returns a random peptide/protein sequence of a length specified by the user. For example 'generate_protein(6)' might return 'WQRTAG'. Your function should:

```
generate_protein <- function(len=9) {
  aa <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T")

  ans <- sample(x=aa, size=len, replace = TRUE)
  paste(ans, collapse = "")
}
```

```
generate_protein(5)
```

```
[1] "RAHML"
```

Generate random protein sequences of length 6 to 13

Q4. Adapt and use your 'generate_protein()' function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function 'for()' or 'sapply()' so that you do not have to call 'generate_protein()' eight times by hand.

```
for(l in 6:13) {  
  cat(">", l, "\n", sep="")  
  cat(generate_protein(l), "\n")  
}
```

```
>6  
YKQVTD  
>7  
MATMITF  
>8  
ADCMWGPY  
>9  
SQSYLKAAA  
>10  
LVHWAACQSY  
>11  
MEACHPENIYW  
>12  
EPNFTKHTGTAF  
>13  
ADIMHQQHPLMAK
```

```
generate_protein(6)
```

```
[1] "ETKSFF"
```

```
generate_protein(7)
```

[1] "VLDALPW"

```
generate_protein(8)
```

[1] "HNPEWWKM"

```
generate_protein(9)
```

[1] "GELYHPLGR"

```
generate_protein(10)
```

[1] "RTSRELKFTG"

```
generate_protein(11)
```

[1] "NCAIQHHEGVG"

```
generate_protein(12)
```

[1] "MWEDHLNVIHCT"

```
generate_protein(13)
```

[1] "FLKIWWEFGYSYV"

BLASTp search against nr — are your peptides “unique in nature”?

Length (aa)	Best hit % identity	Best hit % coverage	Unique? Y/N
6	100%	100%	N
7	100%	100%	N
8	87.50%	100%	Y
9	88.89%	100%	Y
10	100%	90%	Y
11	90%	91%	Y

Length (aa)	Best hit % identity	Best hit % coverage	Unique? Y/N
12	90%	83%	Y
13	100%	77%	Y

Q5a. At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)? [1 pt]

At sequence length 8, the peptides start to look “unique in nature”.

Q5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe. [1 pt]

Very short random peptides are almost always found in nr because their sequence space (20^L) is still small enough that the known protein universe has already sampled a lot of those possibilities by chance. For the longer peptide, it’s not typically always found in nr because it grows exponentially by 20^L . This makes it larger than the total number of sequences in known proteins.

Connecting your findings to immunology

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

I think the minimum length is around 7-8 amino acids because anything below that length, the random peptides are too common in the known protein universe that they aren’t distinctive. It could be a bad design choice for the immune system to present very short peptides because the immune system needs peptides that are distinctive to be able to distinguish self from pathogens. If the presented peptides are too short, then there wouldn’t be enough information and the chance of false matches would increase.